

Gratitude - Server configuration guide

Process for the server configuration team

project_grat

September 11, 2026

Purpose

This document is the setup process for production (or shared staging). Follow the steps in order. The application team delivers the code. The server configuration team provisions hosts, networking, TLS, Postgres, disk and process supervision.

Hosts to provision

- **Database host** - PostgreSQL 16 (14+ OK).
- **Application host** - Node.js 20+, nginx (HTTPS), Node API, static web files and a separate uploads partition or volume.

Reasons for these decisions

Separate uploads partition (20 GB)

Card images and compiled HTML are stored on disk under `api/uploads/`. They are not stored in PostgreSQL. Email and the UI load those files from `/uploads/...` on the public API URL.

A separate partition (or volume) is required so card assets survive OS rebuilds and app redeploys when that partition is left intact. Without it, every imported card image is lost when the application host OS disk is wiped. Back up this partition with the same care as the database.

Current expected load is small (about 6 templates, each about 5 MB or less). 20 GB is enough for that scale with headroom. Grow the uploads partition later if many more cards are added.

A single 60 GB application disk is acceptable if it is split, for example 40 GB for OS/app and 20 GB mounted at the uploads path.

nginx as reverse proxy

nginx terminates HTTPS on port 443, serves the built SPA (single-page application) from `web/dist` and proxies API traffic to the Node process on port 3001.

The Node API must stay on a private port. Exposing 3001 directly would put TLS, static files and API routing on the application process. nginx keeps TLS certificates, SPA routing and the API proxy in one standard entry point for the intranet.

40 GB on the database host

This space is for the operating system, PostgreSQL binaries and `PGDATA`. Application rows (users, sends, audit) are small. 40 GB gives headroom for indexes, WAL (write-ahead log) growth and nightly backup staging on the same host without filling the disk during normal intranet use.

40 GB OS disk on the application host

This space is for the operating system, Node.js, `node_modules` and release artefacts. Card media does not live here. Media uses the separate 20 GB uploads partition so OS and application installs can be replaced without wiping card files.

Sizing (minimum)

Host	CPU / RAM	Disk
Database	2 vCPU / 4 GB	40 GB SSD for Postgres data
Application	2 vCPU / 4 GB	40 GB OS + 20 GB uploads partition (60 GB total OK)

Ports

- **443** - HTTPS to the application host (intranet).
- **3001** - API on localhost or private only. nginx proxies to it (do not expose publicly).
- **5432** - Postgres. Allow only from the application host.

Configuration process

1. Database host

1. Install PostgreSQL 16.
2. Create an empty database (e.g. `project_grat`) and a least-privilege role.
3. Restrict port 5432 to the application host only.
4. Enable nightly backups (retain at least 7 days).
5. Provide the application team the connection string for `DATABASE_URL`.

2. Application host - base

1. Install Node.js 20+ and nginx.
2. Create and mount the uploads partition (20 GB) at the API uploads path (see reasons above).
3. Place the application release on the host (or pull from the repo).
4. Open inbound 443 from the intranet. Keep 3001 private.

3. Application secrets and env

Create `api/.env` from `api/.env.example`. At minimum set:

- `DATABASE_URL` - Postgres connection string from step 1.
- `JWT_SECRET` - strong random secret (not the development default).
- `CORS_ORIGIN` - real HTTPS web origin(s).
- `PUBLIC_API_URL` - public HTTPS API origin (used for card image URLs in email).

Full optional mail/Azure variables are listed in `api/.env.example`.

4. Install, migrate and start API

1. Run `npm ci` in `api/`.
2. Run `npx prisma generate` then `npx prisma migrate deploy`.
3. Do **not** run seed on production unless deliberately resetting demo data.
4. Start the API under systemd (or equivalent): `npx tsx src/index.ts`.
5. Confirm `GET /health` returns OK.

5. Build and serve the web UI

1. Run `npm ci` in `web/`.
2. Build with `VITE_API_URL` set to the public HTTPS API URL.
3. Point nginx at `web/dist` (SPA fallback to `index.html` for client-side routes).
4. Proxy API traffic to port 3001. Ensure `/uploads/` is reachable on the public API origin.
5. Install the TLS certificate on nginx.

6. Handover checks

1. Browser opens the site over HTTPS.
2. Login works (today: development login only. Azure AD is not ready yet).
3. A test upload appears under the uploads partition and loads in the browser.
4. Database and uploads backups are scheduled and verified.

Notes

- No Docker or sample nginx unit is in the repo - write nginx + systemd from this guide.
- Azure AD login is not production-ready yet (`AUTH_MODE=dev` only).
- Treat the uploads partition as critical data, same as the database.